BSCH – AIR

ROS (Robot Operating System)

- Project Documentation –

AIR08 (Team Flame'ingo)

Team Members	
Student Name	**Student Number**
Oguz Sarac	2988989
Emir Sahin	3010269
Berke Yildirim	2972018

Summary of division of work			
Filename / Task	**Oguz Sarac**	**Emir Sahin**	**Berke Yildirim**
Mapping	%100	%0	%0
Localization	%0	%0	%100
Planning	%0	%100	%0
Creating objects	%0	%50	%50
Document	Equally	Equally	Equally

Division of work: <u>work was evenly divided.</u>

Table of Contents:

project_mapping(documentation & screenshots) 3
project_localization (documentation & screenshots).....14
project_planning (documentation & screenshots) 17
Additional Task for Group of 3 - Add additional items to the environment (documentation & screenshots).....21

Documentation & Screen Shots

project_mapping(documentation & screenshots)

Mapping: Section I:

A Script for move the robot around using simple /cmd_vel publishing.

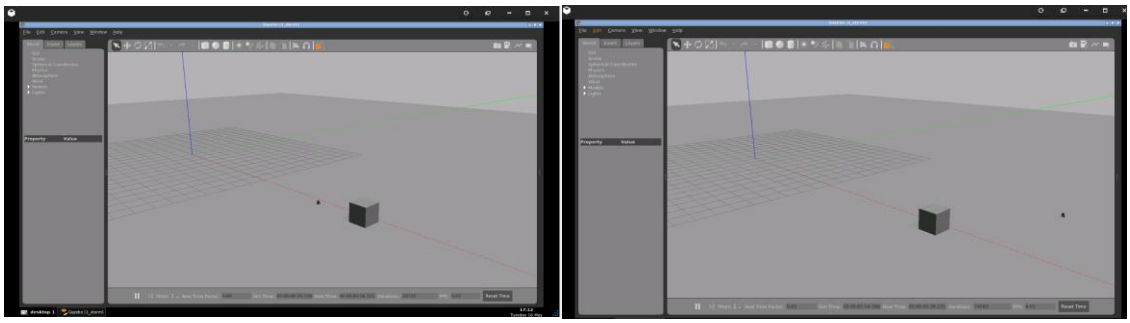
In this section, we will go through the script that we created to move the robot around the environment by using simple *'/cmd_vel'* publishing. The robot will:

- Move at a linear velocity of 0.4.
- If an obstacle is detected within a distance of less than one meter, the robot will turn.
- After turning, it will continue moving at a velocity of 0.4.

Here is a screenshot of the command used to run **turtlebotObjectDetection.launch** file:

```
user:~$ roslaunch project_mapping turtlebotObjectDetection.launch
... logging to /home/user/.ros/log/29c8c66c-f410-11ed-b74a-0242c0a84007/roslaunch-1_xterm-17098.log
g
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

xacro: in-order processing became default in ROS Melodic. You can drop the option.
started roslaunch server http://1_xterm:37679/

SUMMARY
========

PARAMETERS
 * /gazebo/enable_ros_network: True
 * /robot_description: <?xml version="1....
 * /rosdistro: noetic
 * /rosversion: 1.15.11
 * /use_sim_time: True

NODES
  /
    gazebo (gazebo_ros/gzserver)                                                                    ap
    gazebo_gui (gazebo_ros/gzclient)
    spawn_urdf (gazebo_ros/spawn_model)
    turtlebotObjectDetection (project_mapping/turtlebotObjectDetection.py)

ROS_MASTER_URI=http://1_xterm:11311

process[gazebo-1]: started with pid [17299]
process[gazebo_gui-2]: started with pid [17301]
process[spawn_urdf-3]: started with pid [17308]
process[turtlebotObjectDetection-4]: started with pid [17312]
```

Here is a screenshot of the robot's movement after executing our script:



Mapping: Section II:

Creating an RViz Config File

Command: $- rosrun rviz rviz

Before we dive into the creation of an Rviz config file, it's important to understand what Rviz is and the purpose of its configuration file.

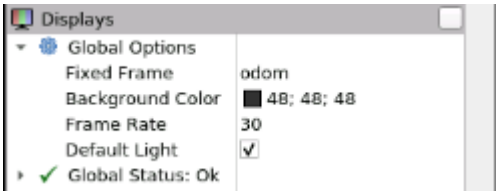
Rviz, which stands for Robot Visualization, is a 3D visualization tool used for debugging and comprehending the state of a robot and its surroundings within the *Robot Operating System* (ROS) framework. This tool is useful for visualizing sensor data, robot models, and trajectories.

In our example, we will save our Rviz config file with the following elements that are linked to appropriate topics.

Setup:

GlobalOptions Display:

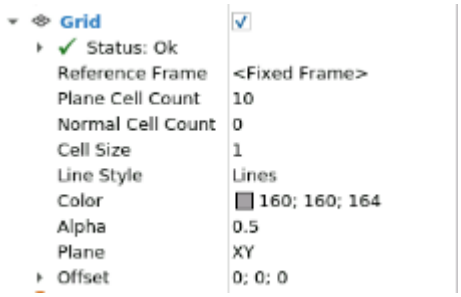
The GlobalOptions display allows you to modify various global options, such as the fixed frame or background color.



- We will need to update the Fixed Frame to ***odom***
- **Tip:** We could update the value of '*Background Color*' if we want to change the background color in Rviz

Grid Display:

The Grid display adds a grid of lines to the Rviz environment.



Grid display has own parameters to help us to customize and re-control the behaviour and UI display of the Grid. Here are some parameters that we could customize and brief information for each:

- **Cell Size:** To specify the size of each cell in the grid display.
- **Line Style:** It offers to different view for the Line. First one is, **Lines** which is thicker than the other Line Style which is the **Billboards**.
- **Color:** To define a color for the cells in the grid display.
- **Alpha:** To set an opacity of a cells in Grid Display.

Map Display:

The Map display is used by robots to navigate using a map-based approach.

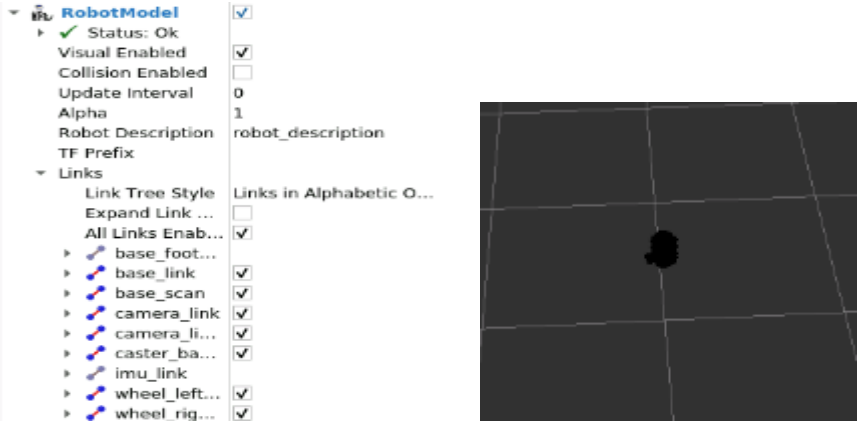
In this representation, a value of **100 corresponds to black**, indicating occupied cells, while a value of **0 corresponds to white**, representing unoccupied cells. **Any other value is represented as grey**, indicating unknown.

As mentioned in the Project Sheet, after executing the node we create with **slam_gmapping** using the launch file, we will be able to find and select the desired **/map** topic.

The value for **Alpha** is adjustable, you could keep test and see which number is fits to your view. The Min and Max values are 0-1.

RobotModel Display:

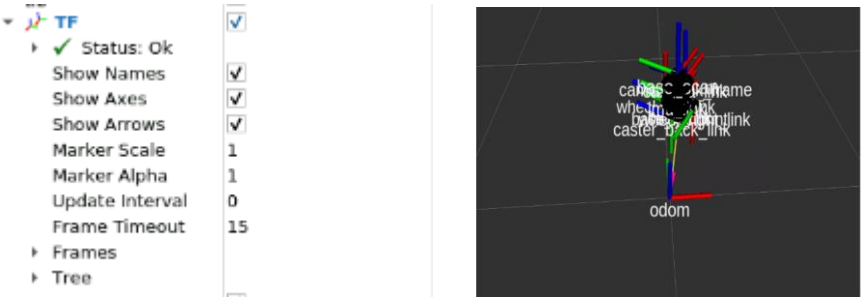
The RobotModel Display in RViz is used to visualize a three-dimensional model of the robot. It provides a graphical representation of the robot's physical components and can be used to validate the design or check for collisions.



Like the previous display, you have the option to adjust the transparency value for the RobotModel Display. The transparency can be set within the range of 0 to 1 in **Alpha**.

TF Display:

The Transform (TF) Display shows the coordinate frames in the Rviz environment.

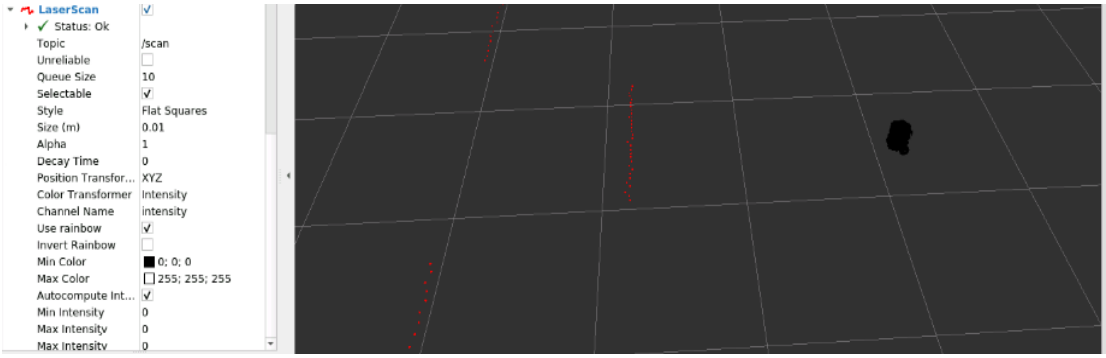


Let's look at some parameters that we can customize along with a brief description of each one:

- **Show Names:** To determine whether or not to display the names of coordinate frames.
- **Show Axes:** To control whether or not to display the coordinate axes of frames.
- **Show Arrows:** To determine whether or not to display arrows indicating of each coordinate frames.

LaserScan Display:

The LaserScan Display allows you to visualize the output of a laser scanner. It is one of the most used displays in Rviz.

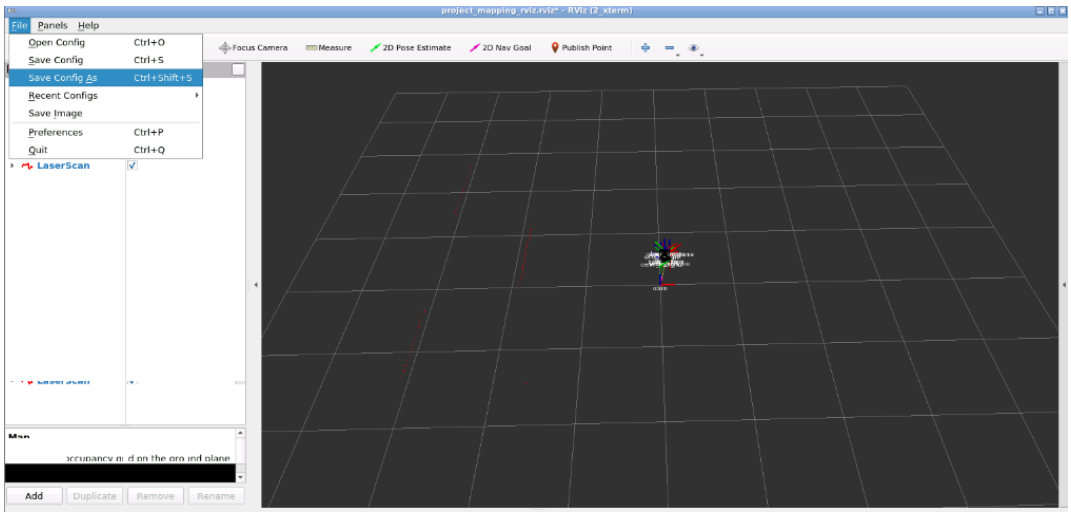


The ***'/scan'*** topic in RViz publishes laser scan data obtained by our robot's laser range finder. This data conveys information about the distances to obstacles in the environment, enabling obstacle detection and mapping.

How can we save our Rviz Config file?

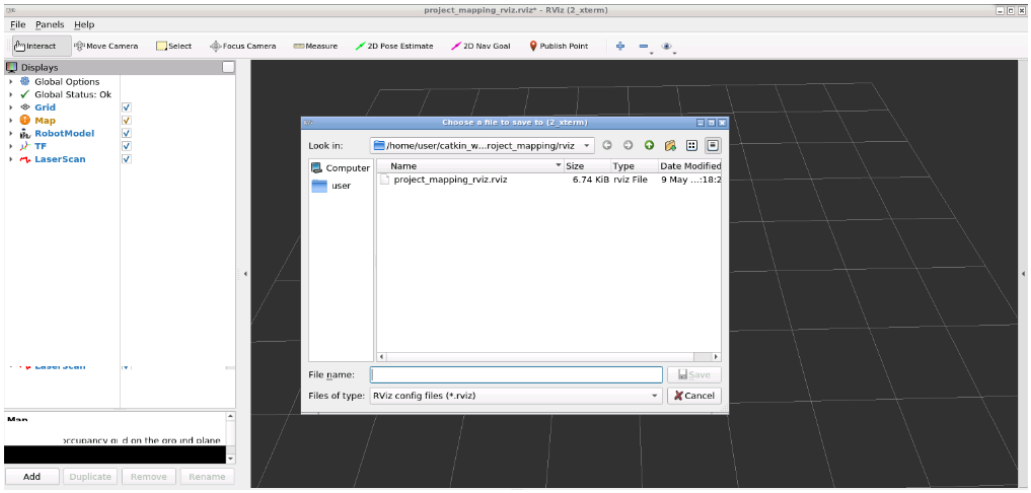
Once we have everything from the previous setup section in the document, we can now save our .rviz file. We have already successfully saved our file, but here are the steps as a guide for new users:

- **Step I:**



Press the File on your top-left of your screen. Then, press the Save Config As (Ctrl+Shift+S).

- **Step II:**



Then, navigate to your package (project_mapping) and go to folder called /rviz. If you do not have a folder called /rviz, you could create by right clicking and choose 'Create a Folder'. Lastly, name your file as project_mapping_rviz.rviz and press 'Save'. That's all! Your Rviz configuration file saved successfully.

Mapping: Section III / IV:

After configuring RViz, we will proceed to create a launch file for the *'slam_gmapping*' node. Additionally, we will update some of the parameters to ensure proper configuration of the *'slam_gmapping'* node. The *'slam_gmapping'* node is configurable and has various parameters that we can update depending on the environment.

In our **project_mapping.launch**, one of the features we have is that instead of running the World, RViz, Robot's Description and URDF, and the slam_gmapping node with separate commands, we have already included them in our launch file. (We referenced the World creation using realrobotlab -> main.launch and Turtlebot's launch files.) To use the teleop (keyboard movement) of our robot and launch the **project_mapping.launch**, all we need to do is run two commands in order: keyboard_teleop and project_mapping.

```
roslaunch project_mapping project_mapping.launch
roslaunch turtlebot_teleop keyboard_teleop.launch
```

To update those parameters, we need a .YAML file called **project_mapping_params.yaml**. In this file, we will include the updated parameters for our project. We will also include this parameter file in our launch file using *'rosparam'*. Here are the files:

Launch File: project_mapping.launch

```
<launch>
  <!-- Arguments -->
  <!-- Define arguments with default values for flexibility -->
  <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type [burger, waffle, waffle_pi]"/>
  <arg name="configuration_basename" default="turtlebot3_lds_2d.lua"/>
  <arg name="open_rviz" default="true"/>
  <arg name="slam_methods" default="gmapping"/>
  <arg name="x_pos" default="0.1"/>
  <arg name="y_pos" default="0.3"/>
  <arg name="z_pos" default="0.05"/>
  <arg name="set_base_frame" default="base_footprint"/>
  <arg name="set_odom_frame" default="odom"/>
  <arg name="set_map_frame"  default="map"/>

  <!-- Launch an empty Gazebo world -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <!-- Specify the world file to be loaded -->
    <arg name="world_name" value="$(find realrobotlab)/worlds/cafe_world.world"/>
    <!-- Set simulation parameters -->
    <arg name="paused" value="false"/>
    <arg name="use_sim_time" value="true"/>
    <arg name="gui" value="true"/>
    <arg name="headless" value="false"/>
    <arg name="debug" value="false"/>
    <arg name="verbose" value="true"/>
  </include>

  <!-- Load robot description -->
  <param name="robot_description" command="$(find xacro)/xacro --inorder $(find
turtlebot3_description)/urdf/turtlebot3_$(arg model).urdf.xacro" />

  <!-- Spawn the robot model in Gazebo -->
  <node pkg="gazebo_ros" type="spawn_model" name="spawn_urdf" args="-urdf -model turtlebot3_$(arg
model) -x $(arg x_pos) -y $(arg y_pos) -z $(arg z_pos) -param robot_description" />

  <!-- TurtleBot3 -->
  <!-- Launch the turtlebot3_bringup package for TurtleBot3 -->
  <include file="$(find turtlebot3_bringup)/launch/turtlebot3_remote.launch">
    <!-- Pass the specified robot model as an argument -->
    <arg name="model" value="$(arg model)" />
  </include>

  <!-- Gmapping -->
  <!-- Run gmapping for SLAM -->
  <node pkg="gmapping" type="slam_gmapping" name="turtlebot3_slam_gmapping" output="screen">
    <!-- Set the base, odometry, and map frames -->
    <param name="base_frame" value="$(arg set_base_frame)"/>
    <param name="odom_frame" value="$(arg set_odom_frame)"/>
    <param name="map_frame"  value="$(arg set_map_frame)"/>
```

Param file: project_mapping_params.yaml

```yaml
map_update_interval: 2.0              # Interval between map updates
maxUrange: 3.0                        # Maximum usable range for laser scan data
particles: 100                        # Number of particles in the particle filter
xmin: -10.0                           # Minimum x-coordinate of the map
ymin: -10.0                           # Minimum y-coordinate of the map
xmax: 10.0                            # Maximum x-coordinate of the map
ymax: 10.0                            # Maximum y-coordinate of the map
```

We already added our comments to the .YAML file but let's look more deeply on each update that we had for parameters:

map_update_interval: | *Default 5.0 -> 2.0* |

We lowered the '*map_update_interval*' to 2 seconds. The main reason for this update is to have a map update more frequently. This would give us a chance to have a better view while we are generating our map.

maxUrange: | *Default 80.0 -> 3.0*|

We updated the '*maxUrange*' value to 3. This parameter is defining the maximum usable range of the laser. It determines the range at which the laser beam is limited or cropped.

Particles: | *Default 30.0 -> 100.0* |

We have updated the '*particles*' value to 100. This adjustment allows for an enhanced accuracy by exploring a larger space of possible poses.

(xmin, ymin, xmax, ymax): | *Default -100.0, 100.0, -100.0, 100.0 -> -10.0, 10.0, -10.0, 10.0*|

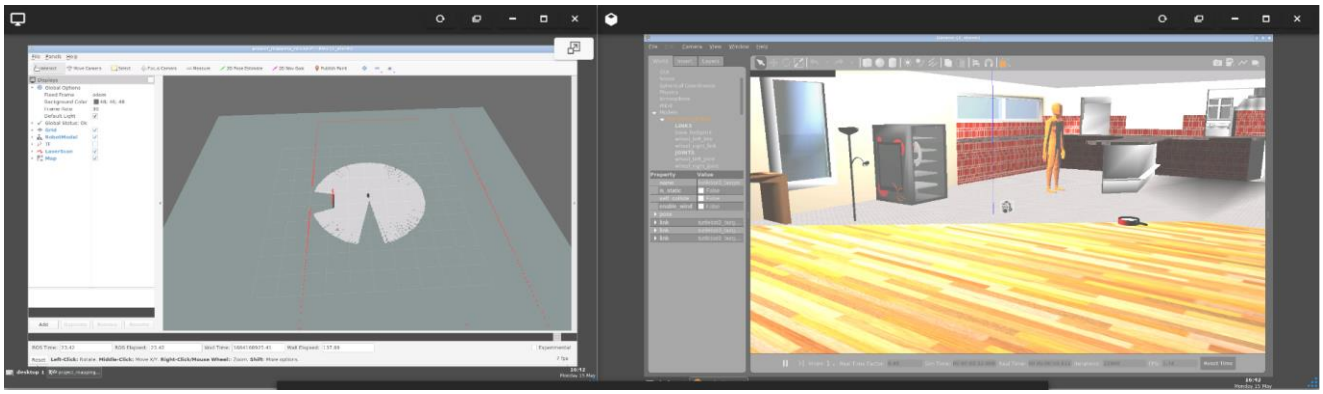
By reducing these values, we limit the area where the SLAM algorithm operates and builds the map. However, a smaller map size can impact performance and accuracy. So, while the updated values may improve accuracy, there may also be limitations due to the restricted map boundaries.

We could list the appropriate topics with the following command:

```
rostopic list
```

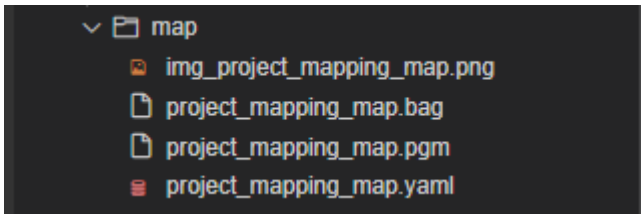
```
user:~$ rostopic list
/clicked_point
/clock
/cmd_vel
/cv_camera/camera_info
/cv_camera/image_raw
/cv_camera/image_raw/compressed
/cv_camera/image_raw/compressed/parameter_descriptions
/cv_camera/image_raw/compressed/parameter_updates
/cv_camera/image_raw/compressedDepth
/cv_camera/image_raw/compressedDepth/parameter_descriptions
/cv_camera/image_raw/compressedDepth/parameter_updates
/cv_camera/image_raw/theora
/cv_camera/image_raw/theora/parameter_descriptions
/cv_camera/image_raw/theora/parameter_updates
/cv_camera/parameter_descriptions
/cv_camera/parameter_updates
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/performance_metrics
/gazebo/set_link_state
/gazebo/set_model_state
/imu
/initialpose
/joint_states
/map
/map_metadata
/map_updates
/move_base_simple/goal
/odom
/rosout
/rosout_agg
/scan
/slam_gmapping/entropy
/tf
/tf_static
```

Lastly, here is a screenshot from mapping process:



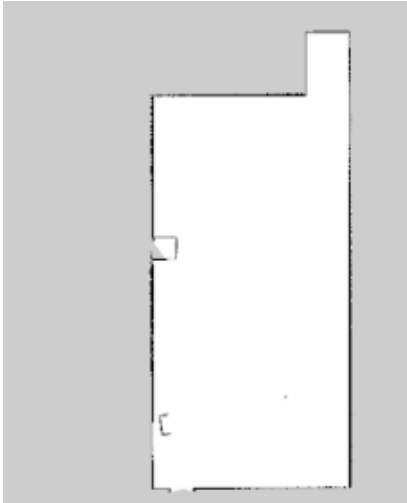
Mapping: Section V:

Once we have successfully saved all our files, they should look like this in the */map* directory:



And here is a converted version of the .pgm file to .png:

(We used GIMP to open the .pgm file. More information about GIMP can be found here: https://www.gimp.org/)



And the parameters of the **project_mapping_map.yaml:**

- **image:** To specify the name of the map image. In our case, it is project_mapping_map.pgm.
- **resolution:** To show resolution of the map, with the size of grid cells.
- **origin:** To define the coordinates of the map's origin x, y and z. It indicates the position of our map in the global coordinate.
- **negate:** Whether the map should/not be negated. If the value of a negate is zero that shows the map is not going to negated.
- **occupied_thresh:** Pixels with an occupancy probability greater than this threshold are considered completely occupied.
- **free_thresh:** The threshold value for considering a grid cell as free. Values below this threshold indicate a free cell.

Mapping: Section VI:

Before we look at the parameters of our **map_provider.launch**, let's have a look at the file itself:

```
<launch>
  <!-- Launch the map_saver node to save the map -->
  <node pkg="map_server"
    type="map_server"
    name="map_server"
    output="screen"
    args="$(find project_mapping)/map/project_mapping_map.yaml">
 </node>
</launch>
```

As we can see, we have created our launch file to provide our map. Now, let's take a look at the parameters in this launch file:

- **pgk=** is the Package of the node.
- **type=** is for specifying the type of the node.
- **name=** is assigning the name.
- **output=** is to output of a node that displayed on our screen.
- **args=** is for passing arguments to our node.

Before we run our launch file, you can open your environment and RViz by using these commands:

```
roslaunch realrobotlab main.launch
rosrun rviz rviz
```

Now, let's launch the following command to check if the node is providing our map:

```
roslaunch project_mapping map_provider.launch
```

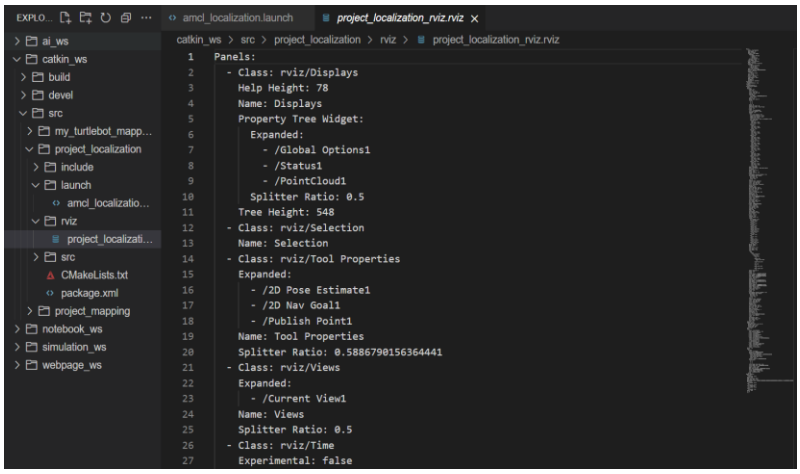
And here is the expected output:

```
user:~$ roslaunch project_mapping map_provider.launch
... logging to /home/user/.ros/log/29c8c66c-f410-11ed-b74a-0242
c0a84007/roslaunch-1_xterm-31620.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://1_xterm:37903/

SUMMARY
========

PARAMETERS
 * /rosdistro: noetic
 * /rosversion: 1.15.11

NODES
  /
    map_server (map_server/map_server)

ROS_MASTER_URI=http://1_xterm:11311

process[map_server-1]: started with pid [31861]
[ INFO] [1684258662.804597864]: Loading map from image "/home/u
ser/catkin_ws/src/project_mapping/map/project_mapping_map.pgm"
[ INFO] [1684258662.947986372, 18.578000000]: Read a 384 X 832
map @ 0.050 m/cell
^[e
```

project_localization (documentation & screenshots)

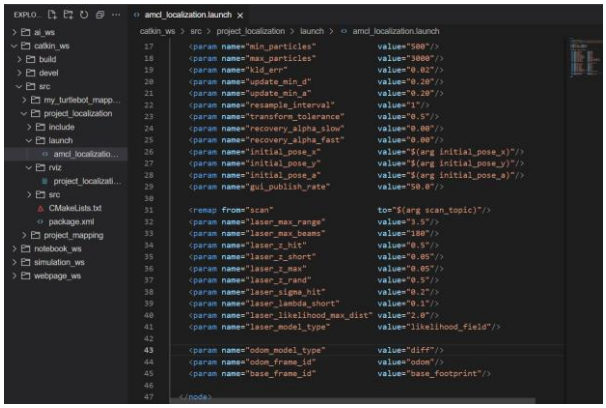
Localization

For the localization we did localize the robot using the amcl node and saved the configuration we used in rviz folder as a project_localization_rviz.rviz.



After saving the configuration, did a launch file that launches the amcl node and the parameters with.

(AMCL, map configuration)

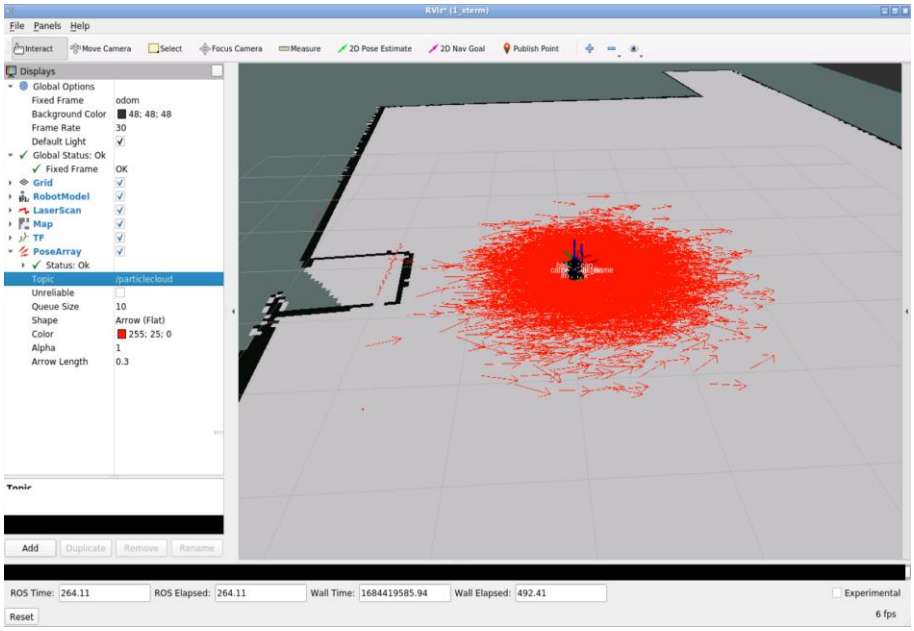


(parameters)

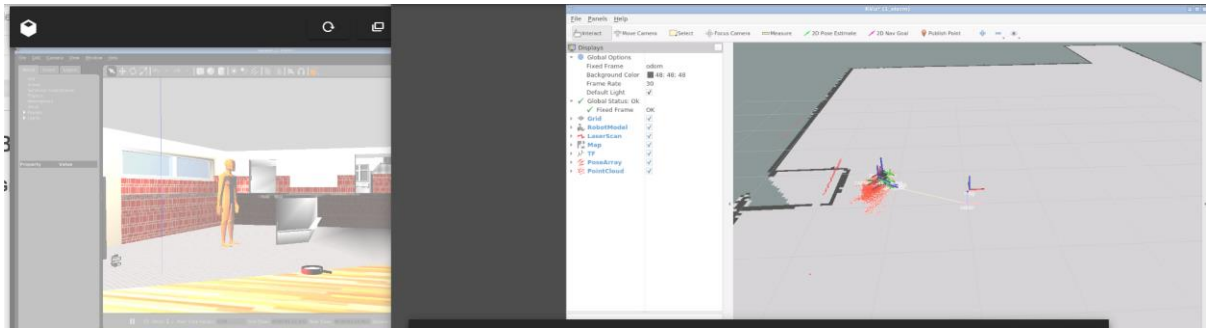
After creating the launch file with the parameters, we start the map_provider_launch command, then the amcl_localization command:

```
user:~/catkin_ws/src/project_localization$ roslaunch launch/amcl_localiza
tion.launch
... logging to /home/user/.ros/log/cbe64b4c-f585-11ed-b779-0242ac1b0007/r
oslaunch-1_xterm-13617.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://1_xterm:40359/

SUMMARY
========

PARAMETERS
 * /rosdistro: noetic
 * /rosversion: 1.15.11

NODES
  /
    amcl (amcl/amcl)
    map_server (map_server/map_server)

ROS_MASTER_URI=http://1_xterm:11311

process[map_server-1]: started with pid [13788]
process[amcl-2]: started with pid [13790]
```

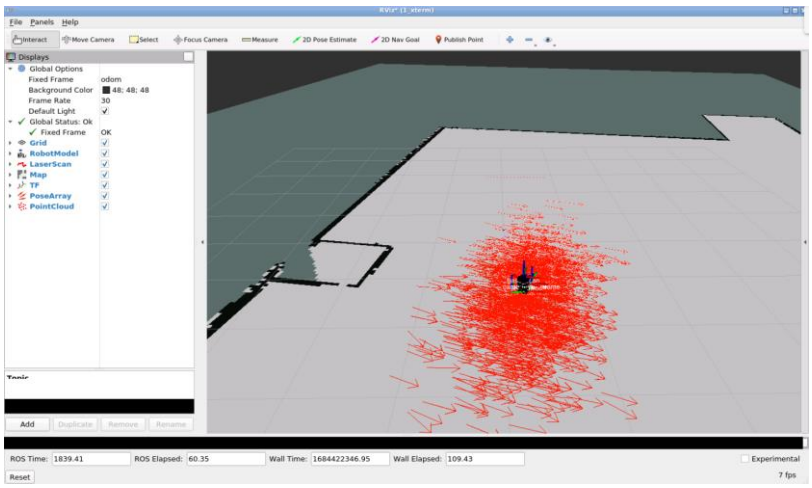
Here is the Rviz configuration after we launch:

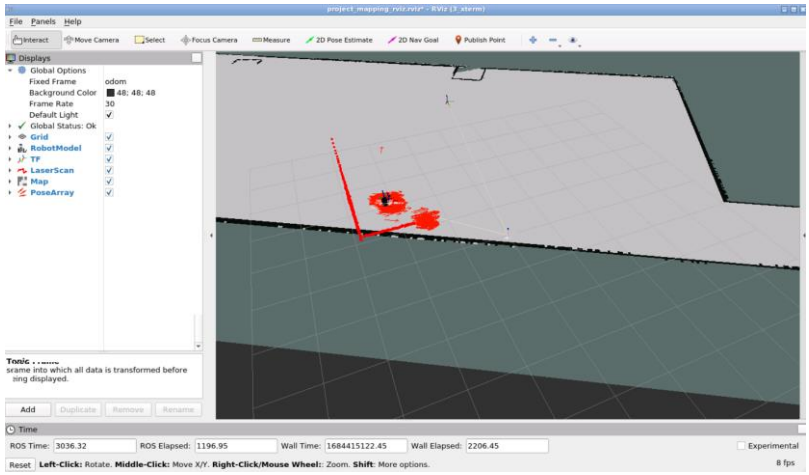


In the Rviz configuration, added the PoseArray and Pointcloud. In the posearray topic we choosed particlecloud so we can see the points of arrows and where it goes.



As we can see the more robot moves, the particle cloud keeps getting smaller. As Dispersed particles and Closer particles.





We used these commands before launching the amcl node:

```
user:~$ roslaunch realrobotlab main.launch
```

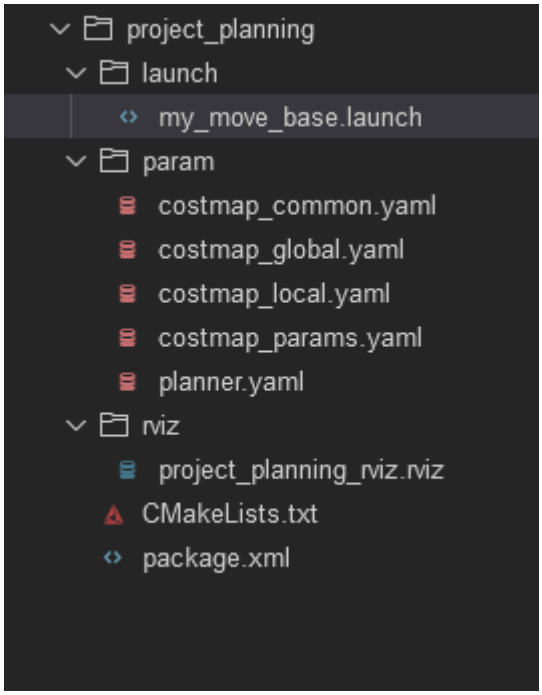
```
^Cuser:~$ rosrun rviz rviz
```

```
user:~$ roslaunch project_mapping map_provider.launch
```

```
user:~/catkin_ws/src/project_localization$ roslaunch launch/amcl_localization.launch
```

project_planning (documentation & screenshots)

Planning



The navfnros, dwaplanner, and costmap parameters, particularly, are configured for the planners in the move_base.launch file. These variables specify the configuration and operation of the navigation system's planning algorithms. The navigation planner uses Dijkstra's or A* algorithms to determine a high-level route from the robot's present location to the desired location. The dwaplanner concentrates on low-level planning and control, producing velocity orders for the robot to follow the course while dynamically dodging obstacles. The environment's representation and the expenses that are allocated to various locations are determined by the costmap parameters. One may fine-tune the robot's planning and obstacle-avoidance skills in the 2D environment by changing these settings.

```
catkin_ws > src > project_planning > launch > my_move_base.launch
<launch>

  <arg name="no_static_map" default="false"/>

  <arg name="base_global_planner" default="navfn/NavfnROS"/>
  <arg name="base_local_planner" default="dwa_local_planner/DWAPlannerROS"/>
  <!-- <arg name="base_local_planner" default="base_local_planner/TrajectoryPlannerROS"/> -->

  <node pkg="move_base" type="move_base" respawn="false" name="move_base" output="screen">

    <param name="base_global_planner" value="$(arg base_global_planner)"/>
    <param name="base_local_planner" value="$(arg base_local_planner)"/>
    <rosparam file="$(find project_planning)/param/planner.yaml" command="load"/>

    <!-- observation sources located in costmap_common.yaml -->
    <rosparam file="$(find project_planning)/param/costmap_common.yaml" command="load" ns="global_costmap" />
    <rosparam file="$(find project_planning)/param/costmap_common.yaml" command="load" ns="local_costmap" />

    <!-- local costmap, needs size -->
    <rosparam file="$(find project_planning)/param/costmap_local.yaml" command="load" ns="local_costmap" />
    <param name="local_costmap/width" value="10.0"/>
    <param name="local_costmap/height" value="10.0"/>

    <!-- static global costmap, static map provides size -->
    <rosparam file="$(find project_planning)/param/costmap_global.yaml" command="load" ns="global_costmap" unless


    <param name="global_costmap/width" value="100.0" if="$(arg no_static_map)"/>
    <param name="global_costmap/height" value="100.0" if="$(arg no_static_map)"/>
  </node>

</launch>
```

Before executing the " move_base" node for planning, the following commands are required to be used.

```
user:~$ roslaunch realrobotlab main.launch
```

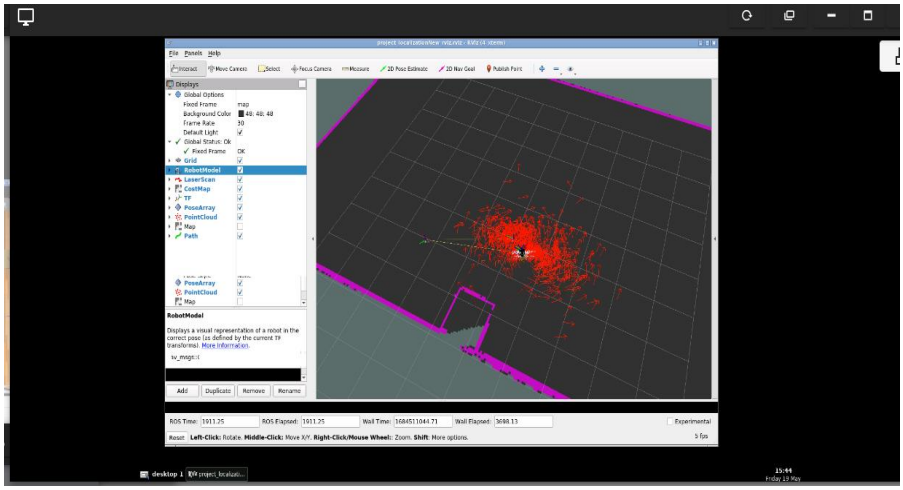
```
^Cuser:~$ rosrun rviz rviz
```

```
user:~$ roslaunch project_mapping map_provider.launch
```

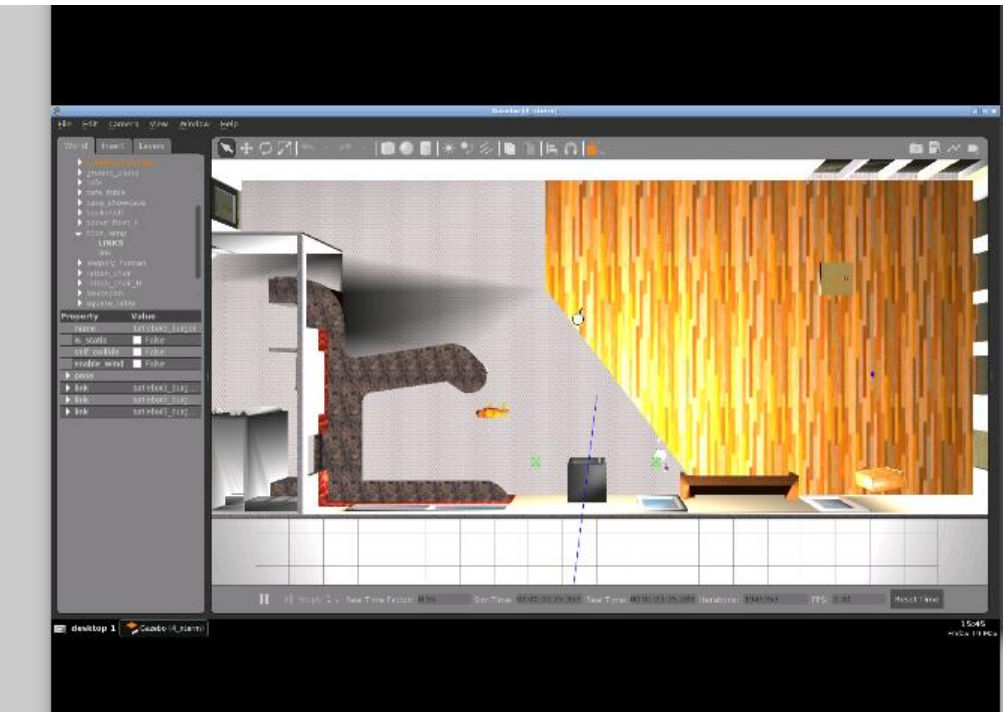
```
user:~/catkin_ws/src/project_localization$ roslaunch launch/amcl_localization.launch
```

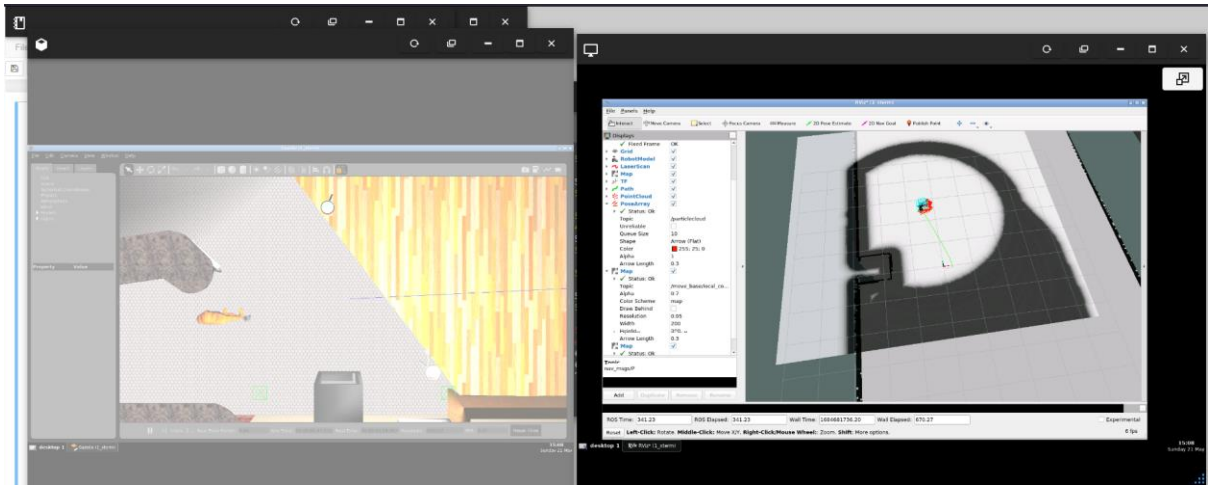
```
user:~$ cd catkin_ws/
user:~/catkin_ws$ cd project_planning
bash: cd: project_planning: No such file or directory
user:~/catkin_ws$ cd src
user:~/catkin_ws/src$ cd project_planning/
user:~/catkin_ws/src/project_planning$ cd launch/
user:~/catkin_ws/src/project_planning/launch$ roslaunch my_move_base.launch
```

After adding map and activating our localization script, we activate our planning node and after that we add path to our rviz. We need to note that if our rviz has no components when we open it we need to add posearray pointcloud to our rviz config for our planning to work.

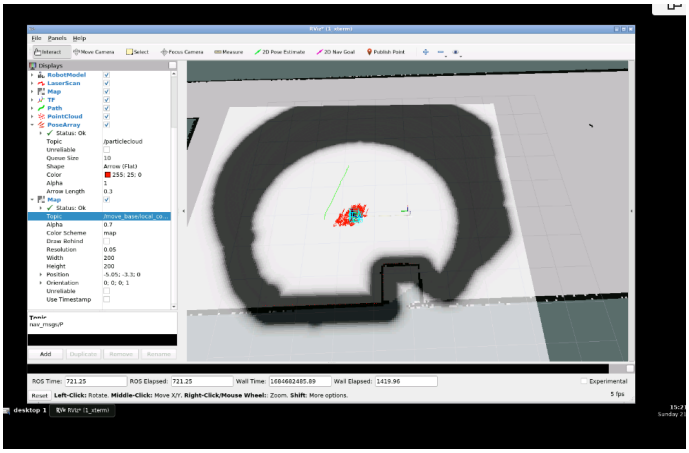


After getting this exact screen we need to put 2d Pose Estimate to our robot's exact location and point the goal point with 2D Nav Goal. Then our robot moves to that exact location with our move_base node. Since our map doesn't content little extras like kitchen counter our robot might not see that obstacle.





After that on rviz section we need to add our localcostmap to see the laser scanned close range map visual.


RViz* (1_xterm)
File Panels Help
Interact
Move Camera
Select
Focus Camera
Measure
2D Pose Estimate
2D Nav Goal
Publish Point
Displays
RobotModel
LaserScan
Map
TF
Path
PointCloud
PoseArray
Status: Ok
Topic /particlecloud
Unreliable
Queue Size 10
Shape Arrow (Flat)
Color 255; 25; 0
Alpha 1
Arrow Length 0.3
Map
Status: Ok
Topic /move_base/local_co...
Alpha 0.7
Color Scheme map
Draw Behind
Resolution 0.05
Width 200
Height 200
Position -5.05; -3.3; 0
Orientation 0; 0; 0; 1
Unreliable
Use Timestamp
Time
nav_msgs/P
Add
Duplicate
Remove
Rename
ROS Time: 721.25
ROS Elapsed: 721.25
Wall Time: 1684682485.89
Wall Elapsed: 1419.96
Experimental
Reset
Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel:: Zoom. Shift: More options.
5 fps
desktop 1
RViz* (1_xterm)

Additional Task for Group of 3 - Add additional items to the environment (documentation & screenshots)

To create the following 5 objects in the Gazebo environment, we will need to run these commands:

```
rosrun gazebo_ros spawn_model -file /home/user/catkin_ws/src/object.urdf -urdf -x 0 -y 0 -z 1 -model my_object
rosrun gazebo_ros spawn_model -file /home/user/catkin_ws/src/object1.urdf -urdf -x 2 -y 2 -z 1 -model my_object1
rosrun gazebo_ros spawn_model -file /home/user/catkin_ws/src/object2.urdf -urdf -x -2 -y 2 -z 1 -model my_object2
rosrun gazebo_ros spawn_model -file /home/user/catkin_ws/src/object3.urdf -urdf -x 2 -y -2 -z 1 -model my_object3
rosrun gazebo_ros spawn_model -file /home/user/catkin_ws/src/object4.urdf -urdf -x -2 -y -2 -z 1 -model my_object4
```

And here is the URDF file for each object that we created:

Object1:

```
<robot name="simple_box">
  <link name="my_box">
    <inertial>
      <origin xyz="2 0 0" />
      <mass value="1.0" />
      <inertia  ixx="1.0" ixy="0.0"  ixz="0.0"  iyy="100.0"  iyz="0.0"  izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="2 0 1"/>
      <geometry>
        <box size="1 1 2" />
      </geometry>
    </visual>
      <collision>
        <origin xyz="2 0 1"/>
        <geometry>
          <box size="1 1 2" />
        </geometry>
      </collision>
  </link>
  <gazebo reference="my_box">
    <material>Gazebo/Blue</material>
  </gazebo>
</robot>
```

Object2:

```
<robot name="simple_box1">
  <link name="my_box1">
    <inertial>
      <origin xyz="0.95 -0.5 0" />
      <mass value="1.0" />
      <inertia  ixx="1.0" ixy="0.0"  ixz="0.0"  iyy="100.0"  iyz="0.0"  izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="0.95 -0.5 0"/>
      <geometry>
        <box size="0.1 0.3 0.2" />
      </geometry>
    </visual>
      <collision>
        <origin xyz="0.95 -0.5 0"/>
        <geometry>
          <box size="0.1 0.3 0.2" />
        </geometry>
      </collision>
  </link>
  <gazebo reference="my_box1">
    <material>Gazebo/Black</material>
  </gazebo>
</robot>
```

Object3:

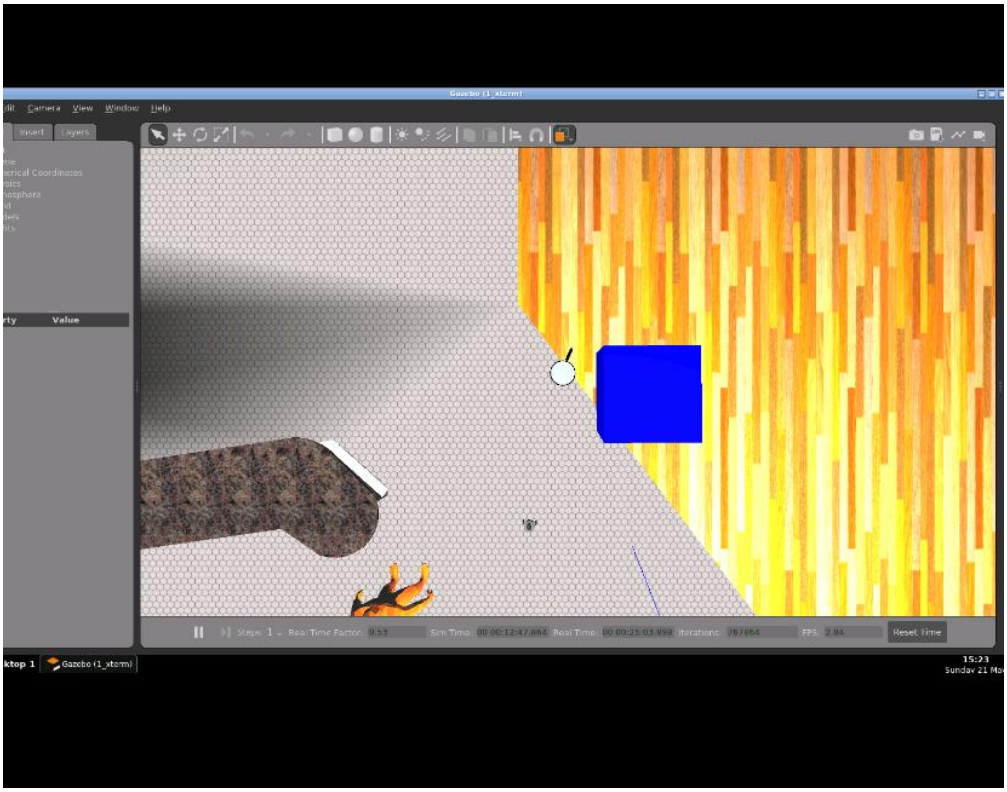
```
<robot name="simple_box2">
  <link name="my_box2">
    <inertial>
      <origin xyz="0.95 -0.5 0" />
      <mass value="1.0" />
      <inertia  ixx="1.0" ixy="0.0"  ixz="0.0"  iyy="100.0"  iyz="0.0"  izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="0.95 -0.5 0"/>
      <geometry>
          <box size="0.4 0.4 0.8" />
      </geometry>
    </visual>
      <collision>
        <origin xyz="0.95 -0.5 0"/>
        <geometry>
          <box size="0.4 0.4 0.8" />
        </geometry>
      </collision>
  </link>
  <gazebo reference="my_box2">
    <material>Gazebo/Red</material>
  </gazebo>
</robot>
```

Object4:

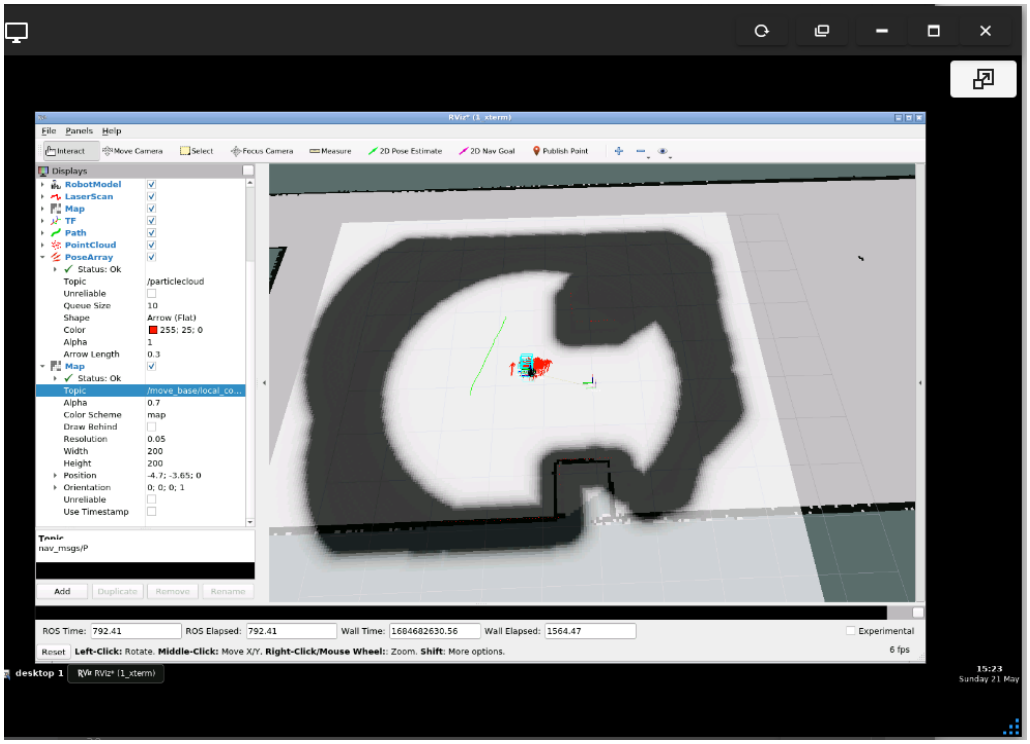
```
<robot name="simple_box3">
  <link name="my_box3">
    <inertial>
      <origin xyz="0.95 -0.5 0" />
      <mass value="1.0" />
      <inertia  ixx="1.0" ixy="0.0"  ixz="0.0"  iyy="100.0"  iyz="0.0"  izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="0.95 -0.5 0"/>
      <geometry>
        <box size="0.1 0.6 0.6" />
      </geometry>
    </visual>
      <collision>
        <origin xyz="0.95 -0.5 0"/>
        <geometry>
          <box size="0.1 0.6 0.6" />
        </geometry>
      </collision>
  </link>
  <gazebo reference="my_box3">
    <material>Gazebo/Yellow</material>
  </gazebo>
</robot>
```

Object5:

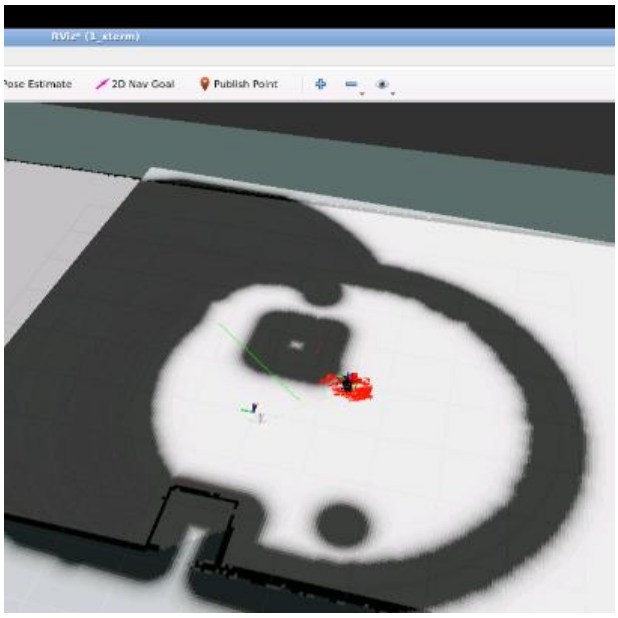
```
<robot name="simple_box4">
  <link name="my_box4">
    <inertial>
      <origin xyz="0.95 -0.5 0" />
      <mass value="1.0" />
      <inertia  ixx="1.0" ixy="0.0"  ixz="0.0"  iyy="100.0"  iyz="0.0"  izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="0.95 -0.5 0"/>
      <geometry>
        <box size="0.1 0.5 0.5" />
      </geometry>
    </visual>
      <collision>
        <origin xyz="0.95 -0.5 0"/>
        <geometry>
          <box size="0.1 0.5 0.5" />
        </geometry>
      </collision>
  </link>
  <gazebo reference="my_box4">
    <material>Gazebo/Green</material>
  </gazebo>
</robot>
```



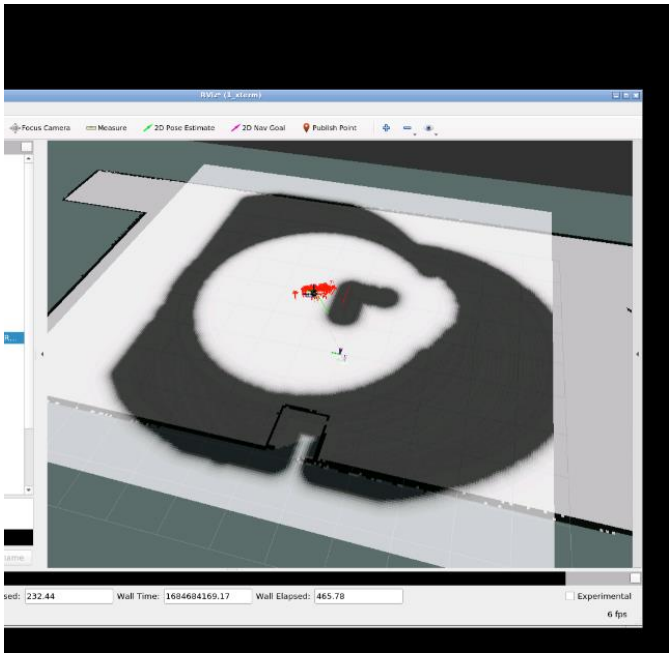
We spawned a giant blue cube next to our robot. Let’s check if that obstacle is seen in our local map.

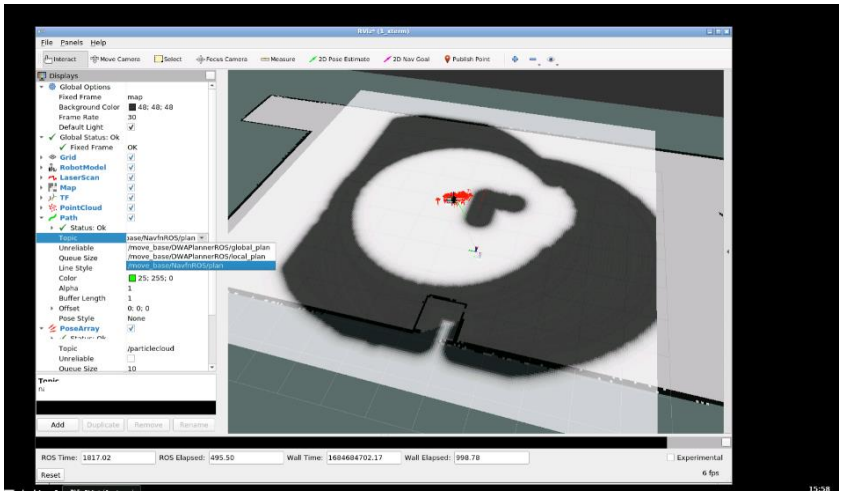


Now that we see our projectile in our map let's try to move our robot through it. After putting our robot into a route that goes through the giant cube, we can see that our robot can create a new plan for avoiding that obstacle.

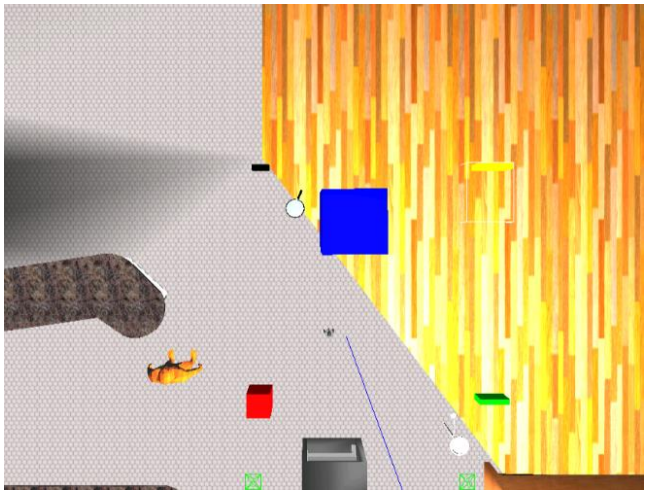



```
[ INFO] [1684684079.573177520, 1510.002000000]: Got new plan
[ INFO] [1684684079.881004625, 1510.196000000]: Got new plan
Warning: TF_REPEATED_DATA ignoring data with redundant timestamp for frame
base_footprint at time 1511.821000 according to authority unknown_publisher
         at line 278 in /tmp/binarydeb/ros-noetic-tf2-0.7.5/src/buffer_core
.cpp
Warning: TF_REPEATED_DATA ignoring data with redundant timestamp for frame
base_footprint at time 1518.728000 according to authority unknown_publisher
         at line 278 in /tmp/binarydeb/ros-noetic-tf2-0.7.5/src/buffer_core
.cpp
[ INFO] [1684684099.562656321, 1520.002000000]: Got new plan
Warning: TF_REPEATED_DATA ignoring data with redundant timestamp for frame
base_footprint at time 1522.945000 according to authority unknown_publisher
         at line 278 in /tmp/binarydeb/ros-noetic-tf2-0.7.5/src/buffer_core
.cpp
Warning: TF_REPEATED_DATA ignoring data with redundant timestamp for frame
base_footprint at time 1523.992000 according to authority unknown_publisher
         at line 278 in /tmp/binarydeb/ros-noetic-tf2-0.7.5/src/buffer_core
.cpp
Warning: TF_REPEATED_DATA ignoring data with redundant timestamp for frame
base_footprint at time 1526.191000 according to authority unknown_publisher
```

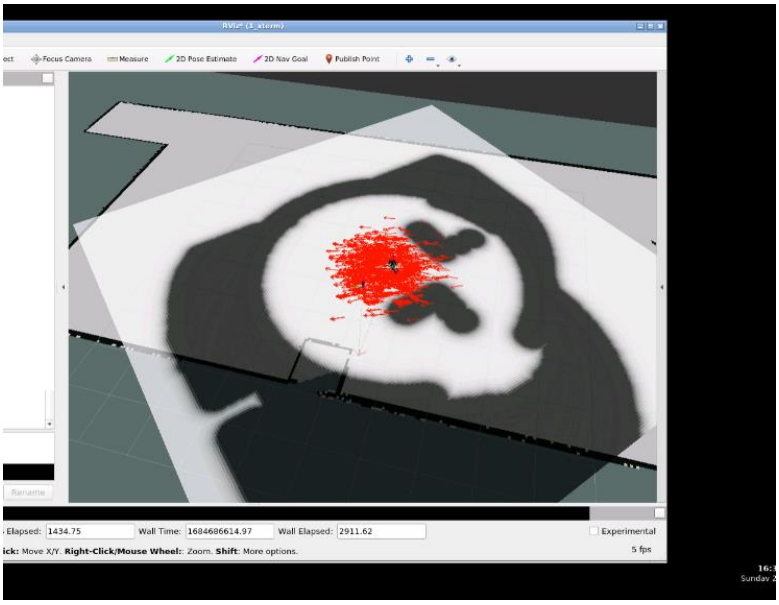




Here are the 5 objects in the environment:



We have 5 additional objects named object, object1, object2, object3, and object4. We can see all the objects in our local map.



1. Costmap Generation

Using the given sensor data, create a costmap as the initial step. The enormous blue cube and other environmental constraints will be described in detail in this costmap. Make sure the costmap generating procedure is set up and performed correctly to appropriately depict the cube's presence.

2. Planner Setup

Choose or use a planner that can use the costmap data to plan a route around barriers. The planner needs to be able to enter the costmap and produce a workable route that stays clear of the enormous blue cube.

3. Accessing the Costmap

Create a link between the costmap and the planner. Make that the planner has real-time access to the costmap data. The enormous blue cube's presence and placement, as well as the expenses related to them, should be detailed in the costmap.

4. Customizing the Costmap Parameters

To make the planning process unique, change the costmap settings. The planner's perception of the costmap and trajectory planning will be influenced by these elements. The following are some crucial variables that may be altered:

Inflation Radius: To create a broader avoidance zone, expand the inflating radius surrounding the enormous blue cube. By doing this, the planner won't draw their pathways too near to the cube.

Cost Scaling: Adapt the huge blue cube's cost scalability factor. Cost increases will cause the planner to view the cube as a higher-cost hurdle, which will result in more cautious path planning.

Resolution: Adjust the costmap grid's resolution to strike a balance between precision and computational effectiveness. Although it could lengthen the design process, a greater resolution enables more accurate cube representation.

To acquire the desired behavior for planning via the huge blue cube, experiment with various parameter values. Adjust the settings iteratively until you get the best result.

5. Path Planning

Run the intended trajectory in the simulation environment once the planner has generated a path. Keep an eye on the robot's progress to ensure that it successfully avoids the large blue cube. We use DWAPlannerROS and NavfnROS.

6. Path Execution

Execute the intended trajectory in the simulation environment when the planner creates a path. Check to see that the robot successfully avoids the large blue cube by keeping an eye on its movements.

7. Iteration and Optimization

Iterate on the costmap parameters and planning procedure if the intended requirements are not reached by the projected trajectory. To avoid the giant blue cube, tweak the settings and restart the planning process until an optimum path is attained.

Alt Note: Don’t forget to add pointcloud, PoseArray, Path, Map (Costmap) to the rviz after fresh start.